

Algorithme d'Évolution Différentielle massivement parallèle sur GPU avec CUDA : comparaison avec la version séquentielle

Zainab Elamri, Omar Elsemry, Mohamed De Franceschi
Master IM – Université de Haute-Alsace (UHA)
Enseignant : Lhassane IDOUMGHAR

Résumé—Nous étudions la parallélisation massive sur GPU de l'Évolution Différentielle (DE/rand/1/bin) à partir d'un code CUDA de PSO fourni par l'enseignant. Nous en dérivons une version séquentielle (CPU) et une version GPU où chaque thread traite un individu, puis nous les comparons au PSO d'origine sur quatre fonctions de test (Sphere, Rastrigin, Rosenbrock, Griewank décalées), pour $\text{Dim} \in \{10, 50, 100\}$, $\text{Pop} \in \{50, 100, 500\}$ et un budget de $10^4 \times \text{Dim}$ évaluations. Sur une NVIDIA RTX 2080 Ti, le DE GPU est jusqu'à 27 fois plus rapide que le DE séquentiel (médiane 3,5), pour une qualité statistiquement équivalente dans 34 configurations sur 36, et il surpasse significativement le PSO de référence dans 34 configurations sur 36. Nous comparons aussi les taux de succès à ceux publiés pour cudaDEi [1], en testant le paramètre $CR = 0,3$ de cet article, qui améliore les grandes populations mais détériore Rosenbrock, et nous discutons les limites liées à la simple précision.

Index Terms—Differential Evolution, GPU, CUDA, massively parallel metaheuristics, global optimization, benchmark functions.

I. INTRODUCTION

Beaucoup de problèmes d'ingénierie se ramènent à la recherche d'un minimum global $x^* = \arg \min_{x \in [x_{\min}, x_{\max}]^D} f(x)$ d'une fonction non convexe, pour laquelle les méthodes à gradient sont inopérantes. Les métaheuristiques de population, comme l'optimisation par essais particuliers (PSO) et l'évolution différentielle (DE) [2], sont des alternatives courantes, mais leur coût croît avec la dimension et la taille de la population.

Ces algorithmes sont naturellement parallèles : les individus subissent les mêmes opérations, et les GPU, qui exécutent des milliers de threads, s'y prêtent bien [1]. L'objectif de ce travail de Master est d'étudier une version massivement parallèle du PSO fournie en CUDA, de la transformer en DE, et de comparer le résultat à une version séquentielle et à la littérature.

Nos contributions sont : (i) une implémentation séquentielle et une implémentation GPU (un thread par individu) de DE/rand/1/bin, (ii) un protocole expérimental reproductible (10 runs, 3 dimensions, 3 tailles de population, 4 fonctions), (iii) une comparaison avec le PSO de référence et, pour la qualité, avec cudaDEi [1]. La section II rappelle la programmation GPU, la section III décrit le DE et nos implémentations, la section IV présente les résultats et la section V conclut.

II. PROGRAMMATION GPU

CUDA est la plateforme de calcul parallèle de NVIDIA. Le programme est hétérogène : le code *hôte* (CPU) alloue la mémoire, copie les données et lance des *noyaux* (`__global__`) qui s'exécutent sur le *device* (GPU). Un lancement `kernel<<blocs, threads>>` crée une grille de blocs de threads ; les threads d'un bloc sont exécutés par groupes de 32 (*warps*) sur un multiprocesseur (SM) selon le modèle SIMT. Les fonctions `__device__` ne sont appelables que depuis le GPU.

La mémoire est hiérarchisée : registres et mémoire locale par thread, mémoire partagée par bloc (rapide), mémoire globale accessible à tous mais de latence élevée. Les transferts hôte $\leftrightarrow$ device passent par le bus PCIe et sont coûteux : une bonne implémentation les minimise. Pour le générateur aléatoire, la bibliothèque cuRAND fournit un état (`curandState`) par thread.

Nos mesures ont été faites sur une NVIDIA GeForce RTX 2080 Ti (architecture Turing, capacité de calcul 7.5, 11 Go) avec CUDA 13.4, compilé pour `sm_75`.

III. ÉVOLUTION DIFFÉRENTIELLE

A. Aperçu de la version séquentielle

La DE [2] fait évoluer une population de N_P vecteurs $x_i \in \mathbb{R}^D$ tirés uniformément dans $[-5,12; 5,12]^D$. À chaque génération, pour chaque individu x_i (le *vecteur cible*) :

- 1) **Mutation** : on tire trois indices distincts $r_1, r_2, r_3 \neq i$ et on forme $v_i = x_{r_1} + F(x_{r_2} - x_{r_3})$.
- 2) **Croisement binomial** : $u_{i,j} = v_{i,j}$ si $\text{rand}_j < CR$ ou $j = j_{\text{rand}}$, sinon $u_{i,j} = x_{i,j}$; j_{rand} garantit qu'au moins une composante vient du mutant. Le résultat est ramené dans les bornes.
- 3) **Sélection** : $x_i \leftarrow u_i$ si $f(u_i) \leq f(x_i)$, sinon x_i est conservé.

Nous utilisons $F = 0,5$ et, selon la série d'expériences, $CR = 0,9$ ou $CR = 0,3$ (valeur de l'article [1]). Les lectures se font dans la population de la génération g et les écritures dans celle de $g+1$ (double buffer), ce qui rend les individus indépendants au sein d'une génération. Le nombre de générations est $G = 10^4 \times D/N_P$, de sorte que le budget

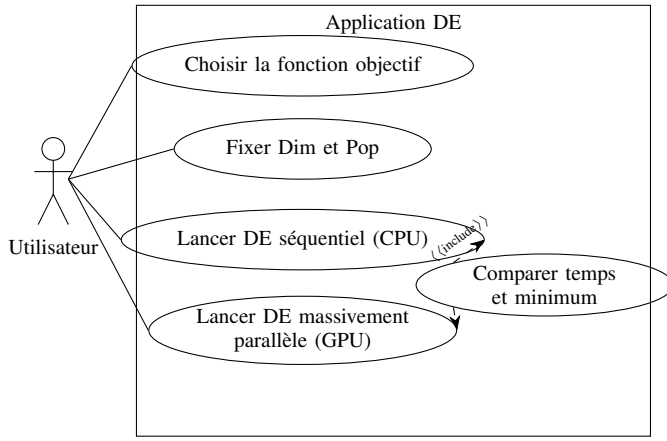


FIGURE 1. Diagramme de cas d'utilisation.

soit de $10^4 \times D$ évaluations. La version séquentielle parcourt les individus dans une boucle et sert de référence.

B. Description de la version massivement parallèle

Le code de PSO fourni utilise un thread par composante de particule et recopie les meilleurs personnels vers le CPU à chaque itération pour mettre à jour le meilleur global. Pour le DE, nous avons choisi un thread par *individu*, car toutes les étapes (mutation, croisement, évaluation, sélection) ne dépendent que de la génération précédente. La population reste sur le GPU pendant toute l'exécution : un seul transfert aller et un seul retour. La signature `cuda_pso` est conservée pour ne pas modifier `main.cpp`. Trois noyaux sont utilisés :

- `kernelInitRandom` : initialise un `curandState` par thread (graine dérivée du `rand()` de l'hôte);
- `kernelEvaluate` : évalue la population initiale;
- `kernelDEGeneration` : pour l'individu $i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$, tire r_1, r_2, r_3 , construit l'essai, l'évalue et applique la sélection en écrivant dans le buffer de sortie.

Les blocs comptent 128 threads. Après chaque génération, l'hôte échange les pointeurs `popIn/popOut`, sans copie. À la fin, l'hôte récupère la population et choisit le meilleur individu. Les figures 1 et 2 présentent la conception en UML (cas d'utilisation, structure des modules) et la figure 3 l'organigramme d'exécution, des appels hôte aux noyaux.

IV. RÉSULTATS EXPÉRIMENTAUX

A. Configuration

Matériel et logiciel. CPU Intel Core i7-8700K (3,7 GHz), GPU NVIDIA RTX 2080 Ti, Arch Linux, pilote 615.71, CUDA 13.4, GCC 15 (-O2).

Algorithmes comparés. (a) DE séquentiel (CPU), (b) DE massivement parallèle (GPU), (c) PSO fourni par l'enseignant (GPU), conservé tel quel ($\omega = 0,5$, $c_1 = c_2 = 1,5$). Le PSO compte des itérations et non des évaluations : nous lui donnons $10^4 \times D / N_P$ itérations, soit le même nombre d'évaluations que le DE.

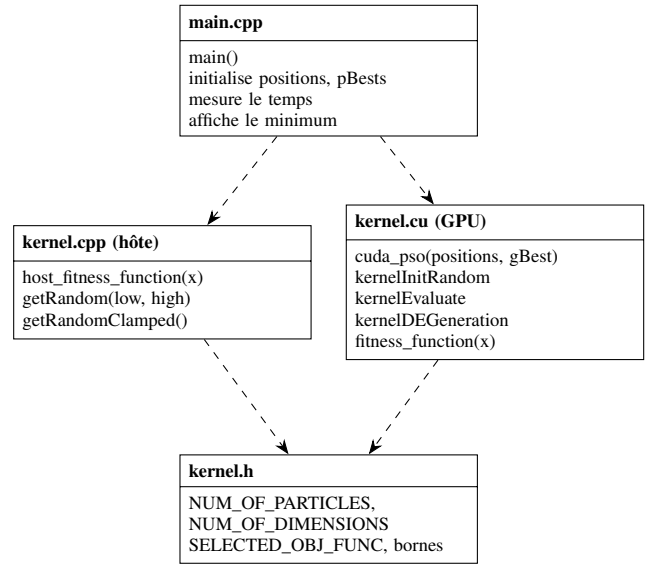


FIGURE 2. Diagramme de classes simplifié (modules du code, dépendances).

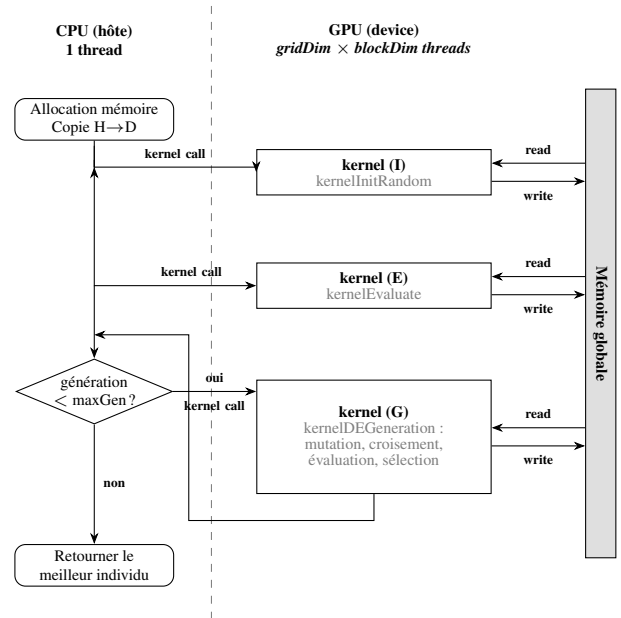


FIGURE 3. Organigramme du DE massivement parallèle (un thread par individu).

Protocole. $\text{Pop} \in \{50, 100, 500\}$, $D \in \{10, 50, 100\}$, budget de $10^4 \times D$ évaluations, 10 exécutions par configuration (graine initiale fixe 12345), soit $3 \times 4 \times 9 \times 10 = 1080$ exécutions. Une seconde série, pour le seul DE (CPU et GPU), reprend le paramètre de l'article, $CR = 0,3$ (720 exécutions supplémentaires); les résultats de la première série ($CR = 0,9$) sont conservés. On rapporte la moyenne de l'erreur (les écarts-types sont dans `results/summary.csv` et servent aux tests statistiques) $\text{EFV} = f(x_{\text{best}}) - f(x^*)$, calculée en double précision à partir de la meilleure solution, et le temps réel moyen. Un run de chauffe (initialisation du contexte

CUDA) est exclu. Le temps GPU inclut allocations, transferts et noyaux. Les tests statistiques sont le test de Wilcoxon de la somme des rangs (seuil 5 %) et le test de Kruskal-Wallis.

B. Fonctions de test

Les quatre fonctions sont décalées, avec $z = x - o$ et un décalage $o = 0$ dans le code fourni, sur le domaine $[-5,12; 5,12]^D$ (les formules sont celles de [1], [3]) :

$$\begin{aligned} f_{\text{Sphere}} &= \sum_{i=1}^D z_i^2 - 450, \\ f_{\text{Rastrigin}} &= \sum_{i=1}^D (z_i^2 - 10 \cos(2\pi z_i) + 10) - 330, \\ f_{\text{Rosenbrock}} &= \sum_{i=1}^{D-1} (100(z_i^2 - z_{i+1})^2 \\ &\quad + (z_i - 1)^2) + 390, \quad z = x + 1, \\ f_{\text{Griewank}} &= \sum_{i=1}^D \frac{z_i^2}{4000} - \prod_{i=1}^D \cos \frac{z_i}{\sqrt{i}} + 1 - 180. \end{aligned}$$

Leur minimum global est atteint en $x = 0$ et vaut respectivement -450 , -330 , 390 et -180 ; l'EFV est la différence à cette valeur. Dans le code fourni, le produit de Griewank était initialisé à 0 au lieu de 1, ce qui annulait le terme en cosinus; nous l'avons corrigé (y compris dans la copie du PSO utilisée comme référence). Les calculs se font en simple précision, avec le biais ajouté à la valeur : la résolution de la fitness autour de -450 est d'environ 3×10^{-5} .

C. Résultats

Les tableaux I à IV donnent la qualité pour toutes les configurations, le tableau V les temps, les figures 4 et 5 les courbes de temps ($D = 10$ et $D = 100$), le tableau VIII l'accélération et le tableau VI les taux de succès.

DE GPU contre DE séquentiel. Les deux versions implémentent le même algorithme avec des générateurs aléatoires différents. Leurs EFV ne diffèrent significativement (Wilcoxon, $p < 0,05$) que dans 2 configurations sur 36, ce qui valide l'implémentation GPU. L'accélération (tableau VIII) varie de 0,47 à 27 avec une médiane de 3,5. Elle augmente avec la population et la dimension, car le GPU dispose de plus de threads : elle est maximale pour Pop = 500. À l'inverse, le GPU est plus lent que le CPU dans 3 configurations (Sphere $D=50$ Pop 50, Rosenbrock $D=50$ Pop 50 et 100). Avec un thread par individu, seuls 50 à 100 threads travaillent et le nombre de générations, donc de lancements de noyaux, est élevé (jusqu'à 2×10^4); la latence de lancement explique vraisemblablement ce résultat, sans que nous l'ayons confirmé par profilage.

DE contre PSO de référence. À budget d'évaluations égal, le DE GPU obtient une EFV moyenne plus faible que le PSO dans 32 configurations sur 36, avec une différence significative dans 34 sur 36. Sur Sphere $D=100$, Pop 100, le DE atteint $3,7 \times 10^{-5}$ contre environ 114 pour le PSO. Le PSO est aussi nettement plus lent (il recopie les meilleurs

personnels vers le CPU à chaque itération). Ses mauvais résultats tiennent probablement en partie à des défauts du code fourni, que nous n'avons pas corrigés : le nombre de blocs est calculé par une division entière, et les tableaux temporaires `tempParticle1/2` sont des variables globales partagées par tous les threads. Il s'agit d'une hypothèse, que nous n'avons pas testée.

Effet de la population. Le budget étant fixe, une grande population signifie peu de générations ($G = 200$ pour $D=10$, Pop 500). Pour Sphere $D=50$, l'EFV passe de $1,4 \times 10^{-5}$ (Pop 100) à 0,21 (Pop 500) : l'algorithme n'a pas convergé. Pour Rastrigin, la meilleure qualité est obtenue avec la plus petite population. Une grande population n'est donc avantageuse que pour le temps, pas pour la qualité.

Taux de succès et comparaison avec la littérature. L'article [1] définit un succès par une EFV $< 10^{-8}$. Avec notre code, ce taux est nul partout, quel que soit CR , y compris sur Sphere (l'article y obtient 1,00) : l'erreur ne descend pas sous environ 10^{-5} à cause de la résolution de la fitness en simple précision avec biais (§IV-B). Nous rapportons donc dans le tableau VI un seuil plus permissif de 10^{-4} , et la comparaison est *favorable à notre code*. Les temps de l'article ne sont pas comparés, le matériel étant différent. Pour Sphere et Griewank, nos taux valent 1,00 (0,90 pour Griewank $D=100$ Pop 50) à $D \in \{50, 100\}$ pour Pop 50 et 100, comme ceux de l'article, mais sont plus bas à $D=10$ pour Griewank (0,20 et 0,80 contre 1,00). Pour Rosenbrock, difficile pour les deux, nous obtenons 1,00 à $D=10$, Pop 100 (0,32 pour l'article) mais 0 pour Pop 50 et 500 (0,44 et 0 pour l'article). Pour Rastrigin, notre DE est moins bon : l'article obtient 1,00 à $D=10$ sur les trois populations, nous 0,50 pour Pop 50 et 0 ensuite. Pour Pop 500, l'article obtient 1,00 sur Sphere et sur Griewank (sauf $D=100$: 0,96), nous 0, du fait du faible nombre de générations.

Effet du paramètre CR (amélioration du DE). L'article utilise $F = 0,5$ et $CR = 0,3$ [1], alors que notre première série utilisait $CR = 0,9$. Nous avons donc relancé le DE (CPU et GPU) avec $CR = 0,3$, tout le reste étant identique. Les tableaux VI et VII comparent les deux réglages. L'effet n'est pas uniforme : l'EFV moyenne du DE GPU est meilleure avec $CR = 0,3$ dans 19 configurations sur 36 et moins bonne dans 17. Les gains concernent les fonctions à structure séparable (ou presque) et surtout les grandes populations, qui disposent de peu de générations : pour Pop 500, Sphere $D=50$ passe de 0,21 à $4,3 \times 10^{-5}$ (taux de succès 0 à 1,00), Griewank $D=50$ et $D=100$ passent de 0 à 1,00. Pour Rastrigin $D=10$, le taux de succès passe de 0,50 et 0 à 1,00 pour Pop 50 et 100, comme dans l'article; il reste nul pour Pop 500 (EFV moyenne 5,7). Griewank $D=10$ passe de 0,20 à 0,90 (Pop 50). En revanche, $CR = 0,3$ détériore Rosenbrock $D=10$, Pop 100 (de 1,00 à 0, EFV moyenne 4,3), et Rastrigin avec Pop 50 (EFV moyenne de 34 à 150 pour $D=50$ et de 95 à 550 pour $D=100$). Une explication plausible, que nous n'avons pas vérifiée, est qu'un CR faible modifie peu de composantes par essai, ce qui aide sur les fonctions séparables mais pénalise Rosenbrock, dont les variables sont couplées. Avec $CR = 0,3$, le DE GPU reste statistiquement équivalent au DE séquentiel

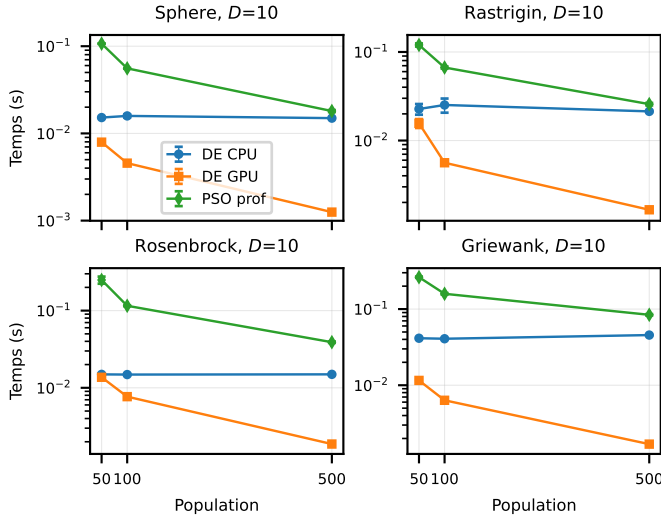


FIGURE 4. Temps d'exécution moyen (échelle logarithmique) en fonction de la population, $D = 10$ (barres : écart-type sur 10 runs).

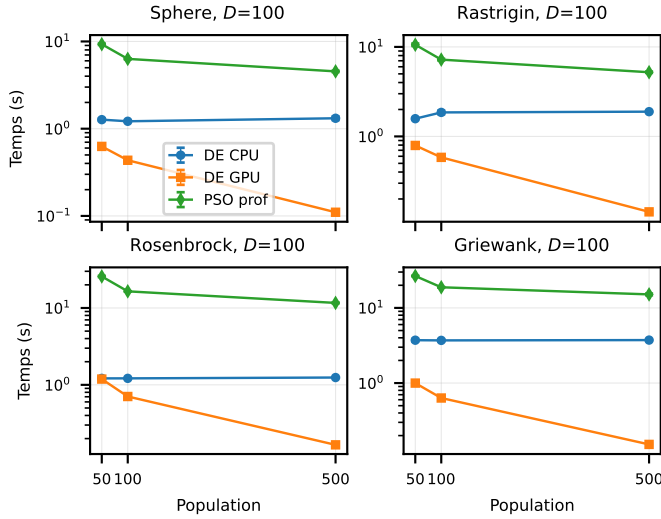


FIGURE 5. Temps d'exécution moyen en fonction de la population, $D = 100$.

(1 configuration sur 36 différente) et l'accélération va de 0,66 à 30 (médiane 4,0). Notre taux de succès se rapproche donc de celui de l'article sur Sphere, Griewank et Rastrigin $D=10$, mais l'écart subsiste sur Rastrigin Pop 500, sur Sphere $D=100$ Pop 500 et sur Griewank $D=10$ Pop 500 ; le nombre de runs (10 contre 25) et la précision de calcul peuvent aussi jouer.

V. CONCLUSION

Nous avons transformé un PSO CUDA en DE/rand/1/bin, implémenté en version séquentielle et en version GPU (un thread par individu), et comparé les deux au PSO d'origine sur quatre fonctions de test, trois dimensions et trois tailles de population. Le DE GPU est statistiquement équivalent au DE séquentiel en qualité, jusqu'à 27 fois plus rapide, et nettement meilleur que le PSO de référence à budget égal. Reprendre le paramètre $CR = 0,3$ de l'article améliore nettement les

TABLE I
SPHERE : EFV MOYENNE (10 RUNS ; PLUS PETIT = MEILLEUR ; $1.2e-5 = 1,2 \times 10^{-5}$).

Dim	Pop	DE CPU	DE GPU	PSO prof
10	50	1.2e-5	1.1e-5	0.791
10	100	1.1e-5	9.9e-6	1.12
10	500	1.0e-5	1.1e-5	0.393
50	50	1.4e-5	1.3e-5	46
50	100	1.3e-5	1.4e-5	33.9
50	500	0.174	0.208	37.1
100	50	3.3e-5	2.5e-5	122
100	100	3.0e-5	3.7e-5	114
100	500	0.173	0.184	133

TABLE II
RASTRIGIN : EFV MOYENNE (10 RUNS ; PLUS PETIT = MEILLEUR ; $1.2e-5 = 1,2 \times 10^{-5}$).

Dim	Pop	DE CPU	DE GPU	PSO prof
10	50	0.959	0.504	23.5
10	100	18.5	17.5	27.5
10	500	28.2	25.2	16.6
50	50	30	33.5	382
50	100	173	243	331
50	500	420	421	325
100	50	99.7	95.4	896
100	100	125	128	864
100	500	894	905	831

TABLE III
ROSENBRICK : EFV MOYENNE (10 RUNS ; PLUS PETIT = MEILLEUR ; $1.2e-5 = 1,2 \times 10^{-5}$).

Dim	Pop	DE CPU	DE GPU	PSO prof
10	50	3.55	3.14	191
10	100	1.1e-5	1.0e-5	311
10	500	3.37	3.12	37.1
50	50	56.5	46.9	4.5e4
50	100	20.4	20.7	2.5e4
50	500	175	164	2.8e4
100	50	137	122	1.2e5
100	100	99.9	111	1.6e5
100	500	207	212	1.4e5

TABLE IV
GRIEWANK : EFV MOYENNE (10 RUNS ; PLUS PETIT = MEILLEUR ; $1.2e-5 = 1,2 \times 10^{-5}$).

Dim	Pop	DE CPU	DE GPU	PSO prof
10	50	7.7e-3	0.0102	0.134
10	100	2.9e-3	5.5e-3	0.173
10	500	0.0324	0.033	0.0324
50	50	6.7e-6	6.2e-6	0.839
50	100	7.0e-6	6.6e-6	0.618
50	500	5.5e-3	6.1e-3	0.701
100	50	8.5e-6	7.5e-6	0.872
100	100	1.2e-5	1.8e-5	0.884
100	500	3.2e-3	3.5e-3	0.881

grandes populations et Rastrigin $D=10$ et rapproche les taux de succès de ceux de cudaDEi, mais détériore Rosenbrock ; l'effet dépend de la fonction.

TABLE V
TEMPS MOYEN (S) SUR LES 4 FONCTIONS (40 RUNS PAR CASE).

Dim	Pop	DE CPU	DE GPU	PSO prof
10	50	0.024	0.012	0.185
10	100	0.024	0.006	0.099
10	500	0.024	0.002	0.042
50	50	0.478	0.389	4.567
50	100	0.499	0.222	3.071
50	500	0.502	0.052	2.092
100	50	1.945	0.902	18.047
100	100	1.996	0.589	12.201
100	500	2.046	0.143	9.128

TABLE VI
TAUX DE SUCCÈS, FORME *article* / *DE GPU* $CR=0,9$ / *DE GPU* $CR=0,3$. ARTICLE [1] : EFV $< 10^{-8}$, 25 RUNS ; NOS VALEURS : EFV $< 10^{-4}$, 10 RUNS.

Fonction	Dim	Pop 50	Pop 100	Pop 500
Sphere	10	1.00/1.00/1.00	1.00/1.00/1.00	1.00/1.00/1.00
Sphere	50	1.00/1.00/1.00	1.00/1.00/1.00	1.00/0.00/1.00
Sphere	100	1.00/1.00/1.00	1.00/1.00/1.00	1.00/0.00/0.00
Rastrigin	10	1.00/0.50/1.00	1.00/0.00/1.00	1.00/0.00/0.00
Rastrigin	50	0.00/0.00/0.00	0.04/0.00/0.00	0.04/0.00/0.00
Rastrigin	100	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
Rosenbrock	10	0.44/0.00/0.00	0.32/1.00/0.00	0.00/0.00/0.00
Rosenbrock	50	0.00/0.00/0.00	0.04/0.00/0.00	0.00/0.00/0.00
Rosenbrock	100	0.00/0.00/0.00	0.00/0.00/0.00	0.00/0.00/0.00
Griewank	10	1.00/0.20/0.90	1.00/0.80/1.00	1.00/0.00/0.00
Griewank	50	0.92/1.00/1.00	1.00/1.00/1.00	1.00/0.00/1.00
Griewank	100	0.84/0.90/1.00	0.96/1.00/1.00	0.96/0.00/1.00

TABLE VII
DE GPU : EFV MOYENNE, FORME $CR=0,9$ / $CR=0,3$ (10 RUNS).

Fonction	Dim	Pop 50	Pop 100	Pop 500
Sphere	10	1.1e-5/1.2e-5	9.9e-6/1.1e-5	1.1e-5/1.2e-5
Sphere	50	1.3e-5/3.9e-5	1.4e-5/4.2e-5	0.208/4.3e-5
Sphere	100	2.5e-5/4.4e-5	3.7e-5/4.8e-5	0.184/5.0e-4
Rastrigin	10	0.504/1.1e-5	17.5/1.1e-5	25.2/5.73
Rastrigin	50	33.5/146	243/173	421/238
Rastrigin	100	95.4/554	128/599	905/719
Rosenbrock	10	3.14/2.65	1.0e-5/4.33	3.12/7.23
Rosenbrock	50	46.9/38.1	20.7/40.3	164/47.2
Rosenbrock	100	122/84.4	111/88.3	212/336
Griewank	10	0.0102/7.5e-4	5.5e-3/6.2e-6	0.033/5.5e-4
Griewank	50	6.2e-6/2.0e-5	6.6e-6/2.0e-5	6.1e-3/2.1e-5
Griewank	100	7.5e-4/2.1e-5	1.8e-5/2.3e-5	3.5e-3/4.7e-5

TABLE VIII
ACCÉLÉRATION T_{CPU}/T_{GPU} DU DE ($CR=0,9$).

Fonction	Dim	Pop 50	Pop 100	Pop 500
Sphere	10	1.9	3.5	12.0
Sphere	50	0.8	1.4	6.1
Sphere	100	2.0	2.8	11.9
Rastrigin	10	1.5	4.5	12.9
Rastrigin	50	1.7	3.0	13.7
Rastrigin	100	2.0	3.2	13.2
Rosenbrock	10	1.1	1.9	8.0
Rosenbrock	50	0.5	0.9	3.7
Rosenbrock	100	1.0	1.7	7.5
Griewank	10	3.6	6.4	27.0
Griewank	50	3.4	5.4	22.1
Griewank	100	3.7	5.8	24.4

thread par individu, le GPU est peu utilisé pour de petites populations, et il peut être plus lent que le CPU ; (iii) le PSO de référence a été gardé tel quel (avec la seule correction de Griewank), donc la comparaison n'est pas à son meilleur niveau ; (iv) pour l'amélioration du DE, nous n'avons testé que $CR = 0,3$ (valeur de l'article), pas d'autres stratégies ni un réglage de F ou de CR par fonction. Perspectives : évaluer la fitness en parallèle sur les dimensions, regrouper les noyaux avec la mémoire partagée comme dans [1], calculer en double précision, et choisir CR selon la fonction ou l'adapter en cours d'exécution (DE/best/1, paramètres auto-adaptatifs).

RÉFÉRENCES

- [1] A. K. Qin, F. Raimondo, F. Forbes, and Y. S. Ong, "An improved CUDA-based implementation of differential evolution on GPU," in *Proc. GECCO'12*, Philadelphia, USA, 2012.
- [2] K. Price, R. Storn, and J. Lampinen, *Differential Evolution : A Practical Approach to Global Optimization*. Springer, 2005.
- [3] P. N. Suganthan *et al.*, "Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization," Tech. Rep., Nanyang Technological University, 2005.

Limites : (i) la fitness en simple précision avec biais empêche d'atteindre le seuil 10^{-8} de l'article ; (ii) avec un